Presentation Layer

Main

+ static main(args: String[]): void

InventoryMenu

- service: ServiceController

+ suppliersSubMenu(): void

Service Layer

ServiceController

- productService: ProductService
- categoryService: CategoryService
- reportService: ReportService
- saleService: SaleService
- orderService: OrderService
- supplierService: SupplierService

ProductService

- domainController: DomainController

+ addProductBatch(...): boolean
+ updateQuantities(...): boolean

CategoryService

- domainController: DomainController

+ addCategory(...): boolean
+ displayCategories(): List<Category>

ReportService

- domainController: DomainController

+ createCategoryReport(category): List<Product>
+ createSubCategoryReport(subCategory): List<Product>
+ createSubSubCategoryReport(size): List<Product>

SaleService

- domainController: DomainController

+ addSale(...): boolean

OrderService

- domainController: DomainController
- supplierSystem: MockSupplierSystem

+ requestShortageOrder(): List<Product>
+ prepareShortageOrder(productId): Order
+ confirmShortageOrder(orderId): boolean
+ markOrderAsReceived(orderId): boolean
+ createPeriodicOrderRule(productId, quantity, dayOfMonth): PeriodicOrderRuleDTO
+ runAutomaticPeriodicOrders(): List<Order>

SupplierService

- supplierSystem: MockSupplierSystem

+ getSuppliers(): List<Supplier>

Supplier Integration - Mock

MockSupplierSystem

- suppliers: List<Supplier>
- agreements: List<SupplyAgreement>
- schedules: List<DeliverySchedule>

- initializeMockData(): void
+ getSuppliers(): List<Supplier>
+ getBestOffer(productId, quantity): SupplierOffer
+ getFixedDeliveryDays(supplierId): List<DayOfWeek>
+ sendOrder(order: Order): boolean

Supplier

- supplierID: String
- supplierName: String

SupplyAgreement

- supplier: Supplier
- productId: String
- minQuantity: int
- unitPrice: double

+ calculatePrice(quantity): double

DeliverySchedule

- supplier: Supplier
- deliveryDay: DayOfWeek
- deliveryTime: LocalTime

SupplierOffer

- supplier: Supplier
- totalPrice: double

Periodic order contract
No extra Domain class

Domain Layer

DomainController

- productRepository: ProductRepository
- categoryRepository: CategoryRepository
- saleRepository: SaleRepository
- orderRepository: OrderRepository
- periodicOrderRuleRepository: PeriodicOrderRuleRepository

+ addCategory(...): boolean
+ addProductBatch(...): boolean
+ getProductsForSystemAlerts(): List<Product>
+ getPeriodicOrderRuleRepository(): PeriodicOrderRuleRepository
+ findOrder(orderId): Order
+ updateOrderStatus(orderId, status): boolean

Sale

- saleID: String
- targetCategory: String
- targetSubCategory: String
- targetBrand: String
- totalRevenue: double
- startDate: LocalDate
- endDate: LocalDate

+ isSaleActive(): boolean

Product

- productID: String
- productName: String
- brand: String
- category: Category
- shopQuantity: int
- warehouseQuantity: int
- minLimit: int
- costPrice: double
- sellPrice: double
- expirationDate: LocalDate
- damagedAmount: int
- isActive: boolean

+ calculateTotalQuantity(): int
+ updatePriceAfterDiscount(sales): void

Category

- categoryName: String
- subCategoryName: String
- subSubCategoryName: String

Order

- orderId: String
- items: List<OrderItem>
- supplierId: String
- supplierName: String
- status: OrderStatus
- orderType: OrderType
- creationDate: LocalDate
- expectedDeliveryDate: LocalDate

+ changeStatus(status: OrderStatus): void
+ calculateTotalPrice(): double

OrderItem

- product: Product
- quantity: int
- unitPrice: double

+ calculateTotalPrice(): double

<<enumeration>>
OrderStatus

DRAFT
SENT
RECEIVED
CANCELLED

<<enumeration>>
OrderType

SHORTAGE
PERIODIC

creates PERIODIC Order

Data Access Layer (SQLite)

SaleRepository

- databaseManager: DatabaseManager

+ add(sale: Sale): boolean
+ findActiveSales(): List<Sale>
+ findAll(): List<Sale>

ProductRepository

- databaseManager: DatabaseManager

+ add(product: Product): boolean
+ update(product: Product): boolean
+ findById(productId: String): Product
+ findByCategory(category): int[], List<Product>
+ findShortageProducts(): List<Product>

CategoryRepository

- databaseManager: DatabaseManager

+ add(category: Category): boolean
+ find(category, subCategory, size): Category
+ findAll(): List<Category>

OrderRepository

- databaseManager: DatabaseManager

+ add(order: Order): boolean
+ findById(orderId: String): Order
+ update(status)(orderId, status): boolean
+ findPeriodicOrders(): List<Order>

PeriodicOrderRuleRepository

- periodicOrderRuleDAO

+ createRule(productId, quantity, dayOfMonth): PeriodicOrderRuleDTO
+ findRulesThatShouldRunToday(): List<PeriodicOrderRuleDTO>
+ advanceNextDeliveryDate(rule): boolean
+ deactivate(ruleId): boolean

Periodic order contract
Stored as SQLite data, not as a new Domain class.

periodic_order_rules:
- ruleId
- productId
- quantity
- dayOfMonth
- nextDeliveryDate
- active

When a rule is due, the system creates a real **Order** with **OrderType.PERIODIC**.

DatabaseManager

- databaseUrl: String
- connection: Connection

+ connect(): Connection
+ createTables(): void
+ createPeriodicOrderRulesTable(): void

DatabaseManager

- databaseUrl: String
- connection: Connection

+ connect(): Connection
+ getConnection(): Connection
+ createTables(): void
+ createPeriodicOrderRulesTable(): void